

gpt

September 27, 2026

0.1 Dataset

Primeiramente, são coletados os textos que serão utilizados

```
[13]: ## Pegando texto para treinamento ##
with open("textos/conto/textoCompleto.txt", "r", encoding="utf-8") as f:
    texto1 = f.read()
f.close()
print("Contos!")

with open("textos/critica/textoCompleto.txt", "r", encoding="utf-8") as f:
    texto2 = f.read()
f.close()
print("Criticas!")
```

Contos!

Criticas!

0.2 Separação de treino e teste

Após pegar todos os textos, deve ser dividido em treino e teste.

Para evitar problemas de contexto, a separação utilizara o número de linhas e a mesma porcentagem será aplicada a cada categoria de textos.

```
[14]: ## Análise dos dados ##
textoTreino = ''
textoTest = ''

for texto in [texto1, texto2]:
    count = texto.count("\n")
    tmp = texto.split("\n")
    textoTreino += '\n'.join(tmp[:int(count*0.75)],)
    textoTest += '\n'.join(tmp[int(count*0.75)+1:])

print("Total de caracteres Treino: ", len(textoTreino))
print("Total de caracteres Teste: ", len(textoTest))
print("Total de linhas Treino: ", textoTreino.count("\n"))

caracteresUnicos = sorted(list(set(textoTreino)))
```

```
print("Caracteres únicos: ", ''.join(caracteresUnicos), "\nTotal de caracteres_
↪únicos: ", len(caracteresUnicos))
```

Total de caracteres Treino: 1349874

Total de caracteres Teste: 442357

Total de linhas Treino: 14650

Caracteres únicos:

```
!"$'()*+,-
.0123456789:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz$ªºÀÁÃÄÇÈÉÊË
ÔÕÚääåãçèéëîðöôõü-'"
```

Total de caracteres únicos: 115

0.3 Codificador/Decodificado

Para a utilização do texto nos modelos, o mesmo precisa ser convertido para algum formato compreensível pelo modelo.

0.3.1 Hot-Enconding

Nesse caso cada caractere é associado a um inteiro e a string é convertida em uma lista de inteiros.

```
[15]: ## Criando o codificador e decodificador ##
def transformar(entrada, padrao):
    '''
    Codifica/Decodifica a entrada baseado no padrão passado.
    '''
    if (type(entrada) == list):
        return ''.join([padrao[c] for c in entrada])
    else:
        return [padrao[c] for c in entrada]

## Dicionarios para one-hot enconding ##
charToIndex, indexToChar = dict(), dict()
for i, ch in enumerate(caracteresUnicos):
    charToIndex[ch] = i
    indexToChar[i] = ch

# print(transformar(textoTreino[:1000], charToIndex))
# print(transformar(transformar(textoTreino[:1000], charToIndex), indexToChar))
```

0.4 Preparando treinamento

Com o codificador pronto, trechos aleatórios do texto de testes serão escolhidos para o treinamento.

O número de trechos é definido por nDeBatch e o tamanho do trecho é definido por contextSize.

```
[18]: import torch
# Define o valor para geração de dados aleatórios (testes iguais)
torch.manual_seed(23092026)
```

```

C = 32 # Números de caracteres utilizados no contexto
B = 10 # Quantidade de amostras

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

def getDataBatchs(tipoDados):
    '''
        Obtém **B** trecho(s) de tamanho **C** da entrada do **tipoDados** definida
    '''
    if tipoDados == "treino":
        dados = torch.tensor(transformar(textoTreino, charToIndex), dtype=torch.
↪long)
    else:
        dados = torch.tensor(transformar(textoTest, charToIndex), dtype=torch.long)
↪# textoTest
    n = torch.randint(0, len(dados) - C, (B,)) #Min, Max e quant. de inteiros
    x = torch.stack([dados[i:i+C] for i in n])
    y = torch.stack([dados[i+1:i+1+C] for i in n])

    # Caso não seja CUDA, remove pin_memory
    return x.pin_memory().to(device, non_blocking=True), y.pin_memory().
↪to(device, non_blocking=True)

```

0.5 Bigram

O primeiro modelo a ser montado é o Bigrama

```

[19]: import torch.nn as nn
class BigramLanguageModel(nn.Module):

    def __init__(self, vocabSize: int):
        super().__init__()
        self.embeddingTable = nn.Embedding(vocabSize, vocabSize)

    def forward(self, x, y=None):
        logits = self.embeddingTable(x)
        batchs, context, vocabSize = logits.shape

        if y == None:
            erro = None
        else:
            # Ajusta a dimensionalidade do logits e do y para o calculo da entropia
            # O calculo de erro espera que o objeto de entrada tenha 2 dimensões:
            # Primeiro o n° de valores previstos
            # Segundo o número de classes/caracteres/tokens
            # Ou seja, batchs*context X vocabSize

```

```

        # O Y é um vetor de comprimento igual ao primeiro valor da entrada com
        ↳ cada valor no range do número de classes/caracteres/tokens
        # Ou seja, será o vetor de comprimento batchs*context com valores de 0 a
        ↳ vocabSize-1
        logits = logits.view(batchs*context, vocabSize).to(device,
        ↳ non_blocking=True)
        y = y.view(batchs*context)
        erro = nn.functional.cross_entropy(logits, y).to(device,
        ↳ non_blocking=True)

        return logits, erro

def generate (self, x, newTokens: int):
    for _ in range(newTokens):
        # Prediz o proximo caractere do contexto C (baseado no ultimo caractere)
        logits, _ = self(x)
        logits = logits[:, -1, :] # (B, C, V) -> (B, V)
        probs = torch.softmax(logits, dim=-1) # (B, V)
        nextChar = torch.multinomial(probs, num_samples=1) # (B, 1)
        x = torch.cat((x, nextChar), dim=1) # (B, C+ 1)
    return x

@torch.no_grad()
def estimate_loss():
    out = []
    model.eval()
    for split in ['treino', 'teste']:
        losses = torch.zeros(EVAL_ITERS)
        for k in range(EVAL_ITERS):
            X, Y = getDataBatches(split)
            logits, loss = model(X, Y)
            losses[k] = loss.item()
        out.append(losses.mean())
    model.train()
    return out

model = BigramLanguageModel(len(caracteresUnicos))
model = model.to(device)

x, y = getDataBatches('treino') # X, Y para treino
logits, loss = model(x, y)
print("Erro: ", loss)

```

Erro: tensor(5.2821, device='cuda:0', grad_fn=<NllLossBackward0>)

Antes de ser treinado o texto gerado são apenas caracteres aleatórios

```
[20]: ## Gerar texto antes de treinar o modelo ##
idx = torch.zeros((1, 1), dtype=torch.long, device=device)
print(transformar(model.generate(idx, 100)[0].tolist(), indexToChar))
```

°-úBÓíóo\$õQé.OnFççnY6à

9!uáôcèdW:9ÔjPÀ"PrNFëfÃÈè°D°'JQç5IÇ0üwaRá'ÈjÓ"Õâ*éétª Í3Çã3jÃÃ°OÔ(Cweg\$ÃînâéY

Loop de treinamento

```
[25]: STEPS = 100
EPOCHES = 10
EVAL_ITERS = 25
for epoche in range(EPOCHES):
    # Criando o Otimizador
    learning_rate=1e-3
    optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)
    for step in range(STEPS):
        # every once in a while evaluate the loss on train and val sets
        if step % EVAL_ITERS == 0:
            losses = estimate_loss()
            print(f"Epohe {epoche}, step {step}: train loss {losses[0]:.4f}, test_
↳loss {losses[1]:.4f}")

            logits, loss = model(x, y)
            optimizer.zero_grad() # Zera os gradientes antes do cálculo dos novos_
↳gradientes
            loss.backward() # Calcula o gradiente atual a partir do erro obtido
            optimizer.step()
```

```
Epohe 0, step 0: train loss 5.4856, test loss 5.2968
Epohe 0, step 25: train loss 5.4766, test loss 5.2540
Epohe 0, step 50: train loss 5.4282, test loss 5.3981
Epohe 0, step 75: train loss 5.3645, test loss 5.2588
Epohe 1, step 0: train loss 5.5034, test loss 5.4696
Epohe 1, step 25: train loss 5.4272, test loss 5.4355
Epohe 1, step 50: train loss 5.3443, test loss 5.2878
Epohe 1, step 75: train loss 5.2247, test loss 5.3497
Epohe 2, step 0: train loss 5.3748, test loss 5.5046
Epohe 2, step 25: train loss 5.2512, test loss 5.3080
Epohe 2, step 50: train loss 5.5219, test loss 5.3452
Epohe 2, step 75: train loss 5.4576, test loss 5.4235
Epohe 3, step 0: train loss 5.3030, test loss 5.3289
Epohe 3, step 25: train loss 5.5270, test loss 5.4050
Epohe 3, step 50: train loss 5.5969, test loss 5.3649
Epohe 3, step 75: train loss 5.3989, test loss 5.4384
Epohe 4, step 0: train loss 5.2659, test loss 5.4009
Epohe 4, step 25: train loss 5.4228, test loss 5.3719
Epohe 4, step 50: train loss 5.3600, test loss 5.3342
```

Epohe 4, step 75: train loss 5.3925, test loss 5.4484
 Epohe 5, step 0: train loss 5.3871, test loss 5.2970
 Epohe 5, step 25: train loss 5.4703, test loss 5.3830
 Epohe 5, step 50: train loss 5.3361, test loss 5.4383
 Epohe 5, step 75: train loss 5.4533, test loss 5.3807
 Epohe 6, step 0: train loss 5.4324, test loss 5.3905
 Epohe 6, step 25: train loss 5.4843, test loss 5.4002
 Epohe 6, step 50: train loss 5.5127, test loss 5.4373
 Epohe 6, step 75: train loss 5.2746, test loss 5.3832
 Epohe 7, step 0: train loss 5.3210, test loss 5.4526
 Epohe 7, step 25: train loss 5.5858, test loss 5.4561
 Epohe 7, step 50: train loss 5.5063, test loss 5.4859
 Epohe 7, step 75: train loss 5.3983, test loss 5.5701
 Epohe 8, step 0: train loss 5.4063, test loss 5.3242
 Epohe 8, step 25: train loss 5.4618, test loss 5.3038
 Epohe 8, step 50: train loss 5.3688, test loss 5.4647
 Epohe 8, step 75: train loss 5.4775, test loss 5.3436
 Epohe 9, step 0: train loss 5.4125, test loss 5.3309
 Epohe 9, step 25: train loss 5.4369, test loss 5.4012
 Epohe 9, step 50: train loss 5.4260, test loss 5.4950
 Epohe 9, step 75: train loss 5.4827, test loss 5.2688

Texto gerado após o treinamento

```
[27]: ## Gerar texto depois de treinar o modelo ##
      idx = torch.tensor([[34]], dtype=torch.long, device=device)
      print(transformar(model.generate(idx, 100)[0].tolist(), indexToChar))
```

Idada e biescom anhesa e ntroio.

- ra mendalo, m dass estação eis ao croulhá m co, cofitere, a eutão

0.6 Head Self-Attention

Para melhorar o resultado obtido, uma forma é a implementação de “cabeças” de atenção para trabalharem em paralelo. Além disso, o uso de embeddings no lugar do mapeamento direto de um caracter para outro.

```
[34]: C = 16 # Números de caracteres utilizados no contexto
      B = 32 # Quantidade de amostras
      E = 32 # Dimensão de embedding
      numberHeads = 4
      H = E//numberHeads # head Size

      import torch.nn as nn
      ## Self Attention ##
      class Head(nn.Module):
          """
          Head of self-attention
          """
```

```

def __init__(self):
    super().__init__()
    self.key = nn.Linear(E, H, bias=False)
    self.query = nn.Linear(E, H, bias=False)
    self.value = nn.Linear(E, H, bias=False)
    self.register_buffer('tril', torch.tril(torch.ones(C, C)))

def forward(self, x):
    b,c,e = x.shape
    k = self.key(x) # (B,C,E) -> (B,C, H)
    q = self.query(x) # (B,C,E) -> (B,C, H)

    # compute attention scores ("affinities")
    wei = q @ k.transpose(-2,-1) * H ** -0.5 # (B, C, H) @ (B, H, C) -> (B, C, C)
    wei = wei.masked_fill(self.tril[:c, :c] == 0, float('-inf')) # (B, C, C)
    wei = torch.nn.functional.softmax(wei, dim=-1) # (B, C, C)
    wei = self.dropout(wei)
    # perform the weighted aggregation of the values
    v = self.value(x) # (B,C,H)
    out = wei @ v # (B, C, C) @ (B, C, H) -> (B, C, H)
    return out

class MultiHead(nn.Module):
    '''Múltiplas Head para atenção em paralelo'''

    def __init__(self, numHead: int):
        super().__init__()
        self.heads = nn.ModuleList([Head() for _ in range(numHead)])

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        return out

class BigramLanguageModel(nn.Module):

    def __init__(self, vocabSize: int):
        super().__init__()
        self.embeddingTable = nn.Embedding(vocabSize, E)
        self.positionEmbeddingTable = nn.Embedding(C, E)
        self.heads = MultiHead(numberHeads)
        self.lm_head = nn.Linear(E, vocabSize)

    def forward(self, x, y=None):
        b, c = x.shape

```

```

tok_emb = self.embeddingTable(x) # (B, C) -> (B, C, E)
pos_emb = self.positionEmbeddingTable(torch.arange(c, device=device)) # (C)
↳ -> (C, E)
x = tok_emb + pos_emb # (B, C, E) + (C, E) -> (B, C, E)
x = self.heads(x) # (B, C, E) -> (B, C, H)
logits = self.lm_head(x) # (B, C, V)

if y == None:
    erro = None
else:
    # Ajusta a dimensionalidade do logits e do y para o calculo da entropia
    # O calculo de erro espera que o objeto de entrada tenha 2 dimensões:
    # Primeiro o n° de valores previstos
    # Segundo o número de classes/caracteres/tokens
    # Ou seja, batchs*context X vocabSize
    # O Y é um vetor de comprimento igual ao primeiro valor da entrada com
    ↳ cada valor no range do número de classes/caracteres/tokens
    # Ou seja, será o vetor de comprimento batchs*context com valores de 0 a
    ↳ vocabSize-1
    b, c, vocabSize = logits.shape
    logits = logits.view(b*c, vocabSize).to(device, non_blocking=True)
    y = y.view(b*c)
    erro = nn.functional.cross_entropy(logits, y).to(device,
    ↳ non_blocking=True)

return logits, erro

def generate (self, x, newTokens: int):
    # X eh (B, C)
    for _ in range(newTokens):
        # Prediz o proximo caractere do contexto C (baseado no ultimo caractere)
        idx = x[:, -C:] # limita aos ultimos tokens
        logits, _ = self(idx)
        logits = logits[:, -1, :] # (B, C, V) -> (B, V)
        probs = torch.softmax(logits, dim=-1) # (B, V)
        nextChar = torch.multinomial(probs, num_samples=1) # (B, 1)
        x = torch.cat((x, nextChar), dim=1) # (B, C+1)
    return x

```

Com isso, pode ser gerado novamente o treinamento e observar os resultados

```

[35]: STEPS = 100
      EPOCHES = 10
      EVAL_ITERS = 100
      # Criando o Otimizador
      learning_rate=1e-3
      optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

```

```

for epoche in range(EPOCHES):
    for step in range(STEPS):
        # every once in a while evaluate the loss on train and val sets
        if step % EVAL_ITERS == 0:
            losses = estimate_loss()
            print(f"Epoche {epoche}, step {step}: train loss {losses[0]:.4f}, test_
↳loss {losses[1]:.4f}")

        logits, loss = model(x, y)
        optimizer.zero_grad() # Zera os gradientes antes do cálculo dos novos_
↳gradientes
        loss.backward() # Calcula o gradiente atual a partir do erro obtido
        optimizer.step()

```

```

Epoche 0, step 0: train loss 5.5112, test loss 5.5012
Epoche 1, step 0: train loss 5.5170, test loss 5.4503
Epoche 2, step 0: train loss 5.5818, test loss 5.5067
Epoche 3, step 0: train loss 5.5220, test loss 5.5086
Epoche 4, step 0: train loss 5.5246, test loss 5.4943
Epoche 5, step 0: train loss 5.4464, test loss 5.5386
Epoche 6, step 0: train loss 5.4993, test loss 5.4704
Epoche 7, step 0: train loss 5.5404, test loss 5.4790
Epoche 8, step 0: train loss 5.5465, test loss 5.4333
Epoche 9, step 0: train loss 5.5420, test loss 5.5266

```

```

[39]: ## Gerar texto depois de treinar o modelo ##
idx = torch.tensor([[34]], dtype=torch.long, device=device)
print(transformar(model.generate(idx, 100)[0].tolist(), indexToChar))

```

IÊÃ2quterapro heção.

E ora ame heveretevera e, e es o oraomeivoicomu ntr qu rocéÓE Por, eialil, onde

0.7 Observação

Toda a implementação apresentada até esse ponto foi baseada no vídeo no link:

<https://www.youtube.com/watch?v=kCc8FmEb1nY&list=PLAqhIrjxibuWI23v9cThsA9GvCAUhRvKZ&index=>

Foram realizadas alterações nos nomes de variáveis, valores de parâmetros, na forma de organização dos hiper parâmetros e na forma de treinamento (épocas e variação no learning rate).

Abaixo tem a versão final inspirada na versão apresentada pelo nano GPT.

Foram incluídas algumas sugestões de otimização, porém a implementação difere do exemplo com as principais diferenças sendo: - Função de ativação mantida como ReLu em vez da GeLu. - Seguindo usando Sequential para a classe Block. - Não foi utilizado os elementos extras para permitir múltiplos modelos serem usados com a mesma estrutura, por padrão, para simplificar e aumentar a legibilidade. - Removi o cálculo de MFU

```
[ ]: import torch
import torch.nn as nn
import numpy as np
import os, pickle, math, time
import inspect
from contextlib import nullcontext

# Define o valor para geração de dados aleatórios (testes iguais)
torch.manual_seed(23092026)
device_type = 'cuda' if torch.cuda.is_available() else 'cpu'
device = torch.device(device_type)

dtype = 'bfloat16' if torch.cuda.is_available() and torch.cuda.
    ↪is_bf16_supported() else 'float16'
ptdtype = {'float32': torch.float32, 'bfloat16': torch.bfloat16, 'float16':
    ↪torch.float16}[dtype]
ctx = nullcontext() if device_type == 'cpu' else torch.amp.
    ↪autocast(device_type=device_type, dtype=ptdtype)

## Self Attention ##
class MultiHead(nn.Module):
    """
    MultiHead otimizado. Todas as Heads computadas utilizando as mesmas
    ↪matrizes.\n
    Tem um melhor desempenho.
    """

    def __init__(self):
        super().__init__()
        # Para otimizacao, as multiHeads pode ser agrupadas e simplificadas para:
        self.attention = nn.Linear(E, 3*E, bias=BIAS)
        self.proj = nn.Linear(E, E)

        self.attentionDropout = nn.Dropout(DROPOUT)
        self.residualDropout = nn.Dropout(DROPOUT)

        self.flash = hasattr(torch.nn.functional, 'scaled_dot_product_attention')
        if not self.flash:
            print("WARNING: Flash Attention requer PyTorch >= 2.0. Versão mais lenta
            ↪utilizada")
            # causal mask to ensure that attention is only applied to the left in the
            ↪input sequence
            self.register_buffer("bias", torch.tril(torch.ones(C, C)).view(1, 1, C,
            ↪C))

    def forward(self, x):
```

```

b,c,e = x.shape
k, q, v = self.attention(x).split(E, dim=2) # (B,C,E)

k = k.view(b, c, numberHeads, H).transpose(1, 2) # (B, nh, C, H)
q = q.view(b, c, numberHeads, H).transpose(1, 2) # (B, nh, C, H)
v = v.view(b, c, numberHeads, H).transpose(1, 2) # (B, nh, C, H)

if self.flash:
    # efficient attention using Flash Attention CUDA kernels
    out = torch.nn.functional.scaled_dot_product_attention(q, k, v,
↪attn_mask=None, dropout_p=DROPOUT if TRAINING else 0, is_causal=True)
else:
    # Implementacao Manual
    wei = q @ k.transpose(-2,-1) * H ** -0.5 # (B, nh, C, H) x (B, nh, H, C)
↪-> (B, nh, C, C)
    wei = wei.masked_fill(self.bias[:, :, :c, :c] == 0, float('-inf')) # (B, nh,
↪C, C)
    wei = nn.softmax(wei, dim=-1) # (B, nh, C, C)
    wei = self.attentionDropout(wei)
    out = wei @ v # (B, nh, C, C) x (B, nh, C, H) -> (B, nh, C, H)

    # Coloca novamente o resultado de todas as Heads em sequencia
    out = out.transpose(1,2).contiguous().view(b,c,E) # (B, nh, C, H) -> (B, C,
↪E)

    # Projecao final
    out = self.proj(out) # (B, C, E) -> (B, C, E)
    out = self.residualDropout(out)
    return out

class FeedFoward(nn.Module):
    '''Linear Layer folowed by non-linearity'''

    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(E, 4 * E, bias=BIAS)
        self.activationFunction = nn.ReLU()
        self.proj = nn.Linear(4 * E, E, bias=BIAS) # Representar o Resíduo e evitar
↪gradient vanishing
        self.dropout = nn.Dropout(DROPOUT)

        self.net = nn.Sequential(
            self.fc,
            self.activationFunction,
            self.proj,
            self.dropout
        )

```

```

def forward(self, x):
    return self.net(x)

class LayerNorm(nn.Module):
    '''
        Responsável pela normalização das **linhas** recebidas.
    '''
    def __init__(self, dim, eps=1e-5):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))
        self.bias = nn.Parameter(torch.zeros(dim)) if BIAS else None

    def forward(self, input):
        return nn.functional.layer_norm(input, self.weight.shape, self.weight, ↵
↵self.bias, self.eps)

class Block(nn.Module):

    def __init__(self):
        super().__init__()
        self.heads = MultiHead()
        self.ffwd = FeedFoward()
        self.ln1 = nn.LayerNorm(E)
        self.ln2 = nn.LayerNorm(E)

    def forward(self, x):
        x = x + self.heads(self.ln1(x)) # (B, C, E) -> (B, C, H*) , H* = E
        x = x + self.ffwd(self.ln2(x)) # (B, C, E)
        return x

class LanguageModel(nn.Module):

    def __init__(self, vocabSize: int):
        super().__init__()

        self.transformer = nn.ModuleDict(dict(
            embeddingTable = nn.Embedding(vocabSize, E),
            positionEmbeddingTable = nn.Embedding(C, E),
            drop = nn.Dropout(DROPOUT),
            # Chamada sequencial do conjunto (SelfAttention -> FeedFoward)
            blocks = nn.ModuleList([Block() for _ in range(BLOCKLAYERS)]),
            ln = LayerNorm(E),
        ))

        self.lm_head = nn.Linear(E, vocabSize, bias=False)

```

```

# Referencia em https://paperswithcode.com/method/weight-tying
self.transformer.embeddingTable.weight = self.lm_head.weight

# Inicializar os pesos do modelo
self.apply(self._init_weights)
# Por causa das camadas residuais, aplicar um inicializador especial para
↪ elas
for pn, p in self.named_parameters():
    if pn.endswith('proj.weight'):
        nn.init.normal_(p, mean=0.0, std=0.02/math.sqrt(2 * BLOCKLAYERS))

print("Numero de parametros: %.2fM" % (self.get_num_params()/1e6,))

@torch.no_grad()
def get_num_params(self, non_embedding=True):
    """
    Retorna o número de parâmetros do modelo.
    """
    n_params = sum(p.numel() for p in self.parameters())
    if non_embedding: # Remove os positional embeddings da contagem
        n_params -= self.transformer.positionEmbeddingTable.weight.numel()
    return n_params

@torch.no_grad()
def _init_weights(self, module):
    '''Inicializa os pesos das funções Lineares e de Embedding'''
    if isinstance(module, nn.Linear):
        nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if module.bias is not None:
            nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        nn.init.normal_(module.weight, mean=0.0, std=0.02)

def configure_optimizers(self, weight_decay, learning_rate, betas,
↪ device_type):
    # Obtem todos os parametros e filtra os que precisam de gradiente
    param_dict = {pn: p for pn, p in self.named_parameters()}
    param_dict = {pn: p for pn, p in param_dict.items() if p.requires_grad}

    # Criar grupos de otimização. Parametros 2D+ terão weight decay, outros não.
    decay_params = [p for _, p in param_dict.items() if p.dim() >= 2]
    nodecay_params = [p for _, p in param_dict.items() if p.dim() < 2]
    optimizationGroups = [
        {'params': decay_params, 'weight_decay': weight_decay},
        {'params': nodecay_params, 'weight_decay': 0.0}
    ]

```

```

    print(f"Parametros com weight decay: {len(decay_params)}, com {sum(p.
↪ numel() for p in decay_params):,} parametros.\nParametros sem weight decay:
↪ {len(nodecay_params)}, com {sum(p.numel() for p in nodecay_params):,}
↪ parametros.")

    # Cria AdamW optimizer e usa a versao fused, se disponivel
    use_fused = ('fused' in inspect.signature(torch.optim.AdamW).parameters)
↪ and (device_type == 'cuda')
    extra_args = dict(fused=True) if use_fused else dict()
    print(f"Possível usar fused AdamW: {use_fused}")

    optimizer = torch.optim.AdamW(otimizationGroups, lr=learning_rate,
↪ betas=betas, **extra_args)
    return optimizer

def forward(self, x, y=None):
    b, c = x.shape

    tok_emb = self.transformer.embeddingTable(x) # (B, C) -> (B, C, E)
    pos_emb = self.transformer.positionEmbeddingTable(torch.arange(c,
↪ dtype=torch.long, device=device)) # (C) -> (C, E)

    x = self.transformer.drop(tok_emb + pos_emb) # (B, C, E) + (C, E) -> (B, C,
↪ E)
    for block in self.transformer.blocks:
        x = block(x) # (B, C, E)
    x = self.transformer.ln(x) # (B, C, E)

    logits = self.lm_head(x) # (B, C, V)

    if y == None:
        erro = None
    else:
        b, c, vocabSize = logits.shape
        logits = logits.view(b*c, vocabSize).to(device)
        y = y.view(b*c)
        erro = nn.functional.cross_entropy(logits, y).to(device)

    return logits, erro

@torch.no_grad()
def generate (self, x, newTokens: int):
    # X eh (B, C)
    for _ in range(newTokens):
        # Prediz o proximo caractere do contexto C (baseado no ultimo caractere)
        idx = x[:, -C:] # limita aos ultimos tokens
        logits, _ = self(idx)

```

```

        logits = logits[:, -1, :] # (B, C, V) -> (B, V)
        probs = torch.softmax(logits, dim=-1) # (B, V)
        nextChar = torch.multinomial(probs, num_samples=1) # (B, 1)
        x = torch.cat((x, nextChar), dim=1) # (B, C+1)
    return x

@torch.no_grad()
def estimate_loss():
    out = []
    model.eval()
    for split in ['treino', 'teste']:
        losses = torch.zeros(EVAL_ITERS)
        for k in range(EVAL_ITERS):
            X, Y = getDataBatchs(split)
            with ctx:
                logits, loss = model(X, Y)
                losses[k] = loss.item()
        out.append(losses.mean())
    model.train()
    return out

# learning rate decay scheduler (cosine with warmup)
def get_lr(it):
    # 1) linear warmup for warmup_iters steps
    if it < warmup_iters:
        return learning_rate * (it + 1) / (warmup_iters + 1)
    # 2) if it > lr_decay_iters, return min learning rate
    if it > lr_decay_iters:
        return min_lr
    # 3) in between, use cosine decay down to min learning rate
    decay_ratio = (it - warmup_iters) / (lr_decay_iters - warmup_iters)
    assert 0 <= decay_ratio <= 1
    coeff = 0.5 * (1.0 + math.cos(math.pi * decay_ratio)) # coeff ranges 0..1
    return min_lr + coeff * (learning_rate - min_lr)

#### Funções auxiliares ####
dataDir = 'data'
outDir = 'treino'

## Get data dos arquivos binarios ##
def getDataBatchs(tipoDados):
    '''
        Obtém **B** trecho(s) de tamanho **C** da entrada do **tipoDados** definida
    '''
    if tipoDados == "treino":
        dados = np.memmap(os.path.join(dataDir, 'train.bin'), dtype=np.uint16,
        ↪mode='r')

```

```

    #dados = torch.tensor(transformar(textoTreino, charToIndex), dtype=torch.
↳ long)
    else:
        dados = np.memmap(os.path.join(dataDir, 'test.bin'), dtype=np.uint16,
↳ mode='r')
        #dados = torch.tensor(transformar(textoTest, charToIndex), dtype=torch.
↳ long) # textoTest
    n = torch.randint(0, len(dados) - C, (B,)) #Min, Max e quant. de inteiros
    x = torch.stack([torch.from_numpy((dados[i:i+C]).astype(np.int64)) for i in
↳ n])
    y = torch.stack([torch.from_numpy((dados[i+1:i+1+C]).astype(np.int64)) for i
↳ in n])

    # Caso não seja CUDA, remove pin_memory
    return x.pin_memory().to(device, non_blocking=True), y.pin_memory().
↳ to(device, non_blocking=True)

## Criando o codificador e decodificador ##
def transformar(entrada, padrao):
    '''
    Codifica/Decodifica a entrada baseado no padrão passado.
    '''
    if (type(entrada) == list):
        return ''.join([padrao[c] for c in entrada])
    else:
        return [padrao[c] for c in entrada]

path = os.path.join(dataDir, 'meta.pkl')
vocabSize = None
if os.path.exists(path):
    with open(path, 'rb') as f:
        meta = pickle.load(f)
        vocabSize = meta['vocab_size']
    print(f"Vocabulário encontrado: {vocabSize} caracteres (inside {path})")

### Hiper parametros ###
# N. de Batches (B)
# Tamanho de Contexto (C)
# N. de Embeddings (E)
# HeadSize (H)
hiperParametros = {
    'C': 256, # Números de caracteres utilizados no contexto
    'B': 64, # Quantidade de amostras
    'E': 384, # Dimensão de embedding
    'numberHeads': 6,
    'blockLayers': 6, # Numero de camadas

```

```

    'dropout': 0.2,
    'bias': True
}
# adamw optimizer
learning_rate = 1e-4 # max learning rate
weight_decay = 1e-1
warmup_iters = 2000
lr_decay_iters = 4000
min_lr = 1e-6

beta1 = 0.9
beta2 = 0.99

TRAINING = True
compile = True
decay_lr = True

iter_num = 0
best_val_loss = 1e9
tipoDeInicio = 'resume'

### Inicializando o modelo ###
print("Inicializando modelo...")
if vocabSize is None:
    raise Exception("Vocabulario nao encontrado!")

if tipoDeInicio == 'zero':
    B, C, E = hiperParametros['B'], hiperParametros['C'], hiperParametros['E']
    numberHeads = hiperParametros['numberHeads']
    H = E//numberHeads # head Size

    BLOCKLAYERS=hiperParametros['blockLayers']
    BIAS=hiperParametros['bias']
    DROPOUT=hiperParametros['dropout']

    model = LanguageModel(vocabSize)

    iter_num = 0
    best_val_loss = 1e3
else:
    print(f"Continuando treinamento...")
    # Continuar treinamento a partir do checkpoint.
    ckpt_path = os.path.join(outDir, 'ckpt.pt')
    checkpoint = torch.load(ckpt_path, map_location=device)
    checkpoint_model_args = checkpoint['hiperParametros']

    for k in ['blockLayers', 'numberHeads', 'B', 'E', 'C', 'bias', 'dropout']:

```

```

    hiperParametros[k] = checkpoint_model_args[k]

B, C, E = hiperParametros['B'], hiperParametros['C'], hiperParametros['E']
numberHeads = hiperParametros['numberHeads']
H = E//numberHeads # head Size

BLOCKLAYERS=hiperParametros['blockLayers']
BIAS=hiperParametros['bias']
DROPOUT=hiperParametros['dropout']

model = LanguageModel(vocabSize)
state_dict = checkpoint['model']

## Erro ja encontrado, mantido por facilidade
# fix the keys of the state dictionary :(
# honestly no idea how checkpoints sometimes get this prefix, have to debug
↪more
unwanted_prefix = '_orig_mod.'
for k,v in list(state_dict.items()):
    if k.startswith(unwanted_prefix):
        state_dict[k[len(unwanted_prefix):]] = state_dict.pop(k)

model.load_state_dict(state_dict)
iter_num = checkpoint['iter_num']
best_val_loss = checkpoint['best_val_loss']

model = model.to(device)

# Otimizador
optimizer = model.configure_optimizers(weight_decay, learning_rate, (beta1,
    ↪beta2), device_type)
if tipoDeInicio != 'zero':
    optimizer.load_state_dict(checkpoint['optimizer'])

checkpoint = None # Libera Memoria

# Compila o modelo
if compile:
    print("Compilando o modelo... (leva aproximadamente 1 minuto)")
    model = torch.compile(model) # requires PyTorch 2.0

x, y = getDataBatches('treino') # X, Y para treino
logits, loss = model(x, y)
print("Erro: ", loss)

MAXSTEPS = 6000
EVAL_ITERS = 100

```

```

raw_model = model
t0 = time.time()
scaler = torch.cuda.amp.GradScaler(enabled=(dtype == 'float16'))

while True:
    lr = get_lr(iter_num) if decay_lr else learning_rate
    for param_group in optimizer.param_groups:
        param_group['lr'] = lr

    # every once in a while evaluate the loss on train and val sets
    if iter_num % EVAL_ITERS == 0:
        losses = estimate_loss()
        # timing and logging
        t1 = time.time()
        dt = t1 - t0
        t0 = t1
        print(f"Step {iter_num}: Train loss: {losses[0]:.4f}, Test loss: {losses[1]:
↪.4f}, Tempo: {dt*1000:.2f} ms, lr: {lr}")

        if losses[1] < best_val_loss:
            best_val_loss = losses[1]
            if iter_num > 0:
                checkpoint = {
                    'model': raw_model.state_dict(),
                    'optimizer': optimizer.state_dict(),
                    'hiperParametros': hiperParametros,
                    'iter_num': iter_num,
                    'best_val_loss': best_val_loss,
                }
                print(f"Salvando o checkpoint {outDir}")
                torch.save(checkpoint, os.path.join(outDir, 'ckpt.pt'))

    with ctx:
        logits, loss = model(x, y)

    x, y = getDataBatches('treino')
    # backward pass, with gradient scaling if training in fp16
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
    optimizer.zero_grad(set_to_none=True)

    iter_num += 1

    if iter_num > MAXSTEPS:
        break

```

```

## Gerar texto após treinar o modelo ##
# idx = torch.tensor([[0]], dtype=torch.long, device=device)
# print(transformar(model.generate(idx, 1000)[0].tolist(), indexToChar))

```

Para gerar o texto com o modelo gerado e salvo no checkpoint, pode ser feito com o código abaixo

```

[1]: import os, math
import pickle
from contextlib import nullcontext
import torch
import torch.nn as nn
import numpy as np

# -----
init_from = 'resume' # either 'resume' (from an out_dir) or a gpt2 variant (e.g.
↳ 'gpt2-xl')
outDir = 'treino' # ignored if init_from is not 'resume'
start = "\n" # or "</endoftext/>" or etc. Can also specify a file, use as:
↳ "FILE:prompt.txt"
num_samples = 4 # number of samples to draw
max_new_tokens = 500 # number of tokens generated in each sample
temperature = 0.8 # 1.0 = no change, < 1.0 = less random, > 1.0 = more random,
↳ in predictions
top_k = 200 # retain only the top_k most likely tokens, clamp others to have 0
↳ probability
seed = 1337
# -----

device_type = 'cuda' if torch.cuda.is_available() else 'cpu'
device = torch.device(device_type)

dtype = 'bfloat16' if torch.cuda.is_available() and torch.cuda.
↳ is_bf16_supported() else 'float16'
ptdtype = {'float32': torch.float32, 'bfloat16': torch.bfloat16, 'float16':
↳ torch.float16}[dtype]
ctx = nullcontext() if device_type == 'cpu' else torch.amp.
↳ autocast(device_type=device_type, dtype=ptdtype)

## Self Attention ##
class MultiHead(nn.Module):
    """
    MultiHead otimizado. Todas as Heads computadas utilizando as mesmas
    ↳ matrizes.
    Tem um melhor desempenho.
    """

    def __init__(self):

```

```

super().__init__()
# Para otimizacao, as multiHeads pode ser agrupadas e simplificadas para:
self.attention = nn.Linear(E, 3*E, bias=BIAS)
self.proj = nn.Linear(E, E)

self.attentioDropout = nn.Dropout(DROPOUT)
self.residualDropout = nn.Dropout(DROPOUT)

self.flash = hasattr(torch.nn.functional, 'scaled_dot_product_attention')
if not self.flash:
    print("WARNING: Flash Attention requer PyTorch >= 2.0. Versão mais lenta,
utilizada")
    # causal mask to ensure that attention is only applied to the left in the
input sequence
    self.register_buffer("bias", torch.tril(torch.ones(C, C)).view(1, 1, C,
C))

def forward(self, x):
    b,c,e = x.shape
    k, q, v = self.attention(x).split(E, dim=2) # (B,C,E)

    k = k.view(b, c, numberHeads, H).transpose(1, 2) # (B, nh, C, H)
    q = q.view(b, c, numberHeads, H).transpose(1, 2) # (B, nh, C, H)
    v = v.view(b, c, numberHeads, H).transpose(1, 2) # (B, nh, C, H)

    if self.flash:
        # efficient attention using Flash Attention CUDA kernels
        out = torch.nn.functional.scaled_dot_product_attention(q, k, v,
attn_mask=None, dropout_p=0, is_causal=True)
    else:
        # Implementacao Manual
        wei = q @ k.transpose(-2,-1) * H ** -0.5 # (B, nh, C, H) x (B, nh, H, C)
        -> (B, nh, C, C)
        wei = wei.masked_fill(self.bias[:, :, :c, :c] == 0, float('-inf')) # (B, nh,
C, C)
        wei = nn.softmax(wei, dim=-1) # (B, nh, C, C)
        wei = self.attentioDropout(wei)
        out = wei @ v # (B, nh, C, C) x (B, nh, C, H) -> (B, nh, C, H)

    # Coloca novamente o resultado de todas as Heads em sequencia
    out = out.transpose(1,2).contiguous().view(b,c,E) # (B, nh, C, H) -> (B, C,
E)

    # Projecao final
    out = self.proj(out) # (B, C, E) -> (B, C, E)
    out = self.residualDropout(out)

```

```

        return out

class FeedFoward(nn.Module):
    '''Linear Layer folowed by non-linearity'''

    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(E, 4 * E, bias=BIAS)
        self.activationFunction = nn.ReLU()
        self.proj = nn.Linear(4 * E, E, bias=BIAS) # Representar o Resíduo e evitar
        ↪ gradient vanishing
        self.dropout = nn.Dropout(DROPOUT)

        self.net = nn.Sequential(
            self.fc,
            self.activationFunction,
            self.proj,
            self.dropout
        )

    def forward(self, x):
        return self.net(x)

class LayerNorm(nn.Module):
    '''
        Responsável pela normalização das "**linhas**" recebidas.
    '''

    def __init__(self, dim, eps=1e-5):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))
        self.bias = nn.Parameter(torch.zeros(dim)) if BIAS else None

    # def __call__(self, x):
    #     # calculate the forward pass
    #     xmean = x.mean(1, keepdim=True) # batch mean
    #     xvar = x.var(1, keepdim=True) # batch variance
    #     xhat = (x - xmean) / torch.sqrt(xvar + self.eps) # normalize to unit
    ↪ variance
    #     self.out = self.gamma * xhat + self.beta
    #     return self.out
    # def parameters(self):
    #     return [self.gamma, self.beta]

    def forward(self, input):
        return nn.functional.layer_norm(input, self.weight.shape, self.weight,
        ↪ self.bias, self.eps)

```

```

class Block(nn.Module):

    def __init__(self):
        super().__init__()
        self.heads = MultiHead()
        self.ffwd = FeedFoward()
        self.ln1 = nn.LayerNorm(E)
        self.ln2 = nn.LayerNorm(E)

    def forward(self, x):
        x = x + self.heads(self.ln1(x)) # (B, C, E) -> (B, C, H*) , H* = E
        x = x + self.ffwd(self.ln2(x)) # (B, C, E)
        return x

class LanguageModel(nn.Module):

    def __init__(self, vocabSize: int):
        super().__init__()

        self.transformer = nn.ModuleDict(dict(
            embeddingTable = nn.Embedding(vocabSize, E),
            positionEmbeddingTable = nn.Embedding(C, E),
            drop = nn.Dropout(DROPOUT),
            # Chamada sequencial do conjunto (SelfAttention -> FeedFoward)
            blocks = nn.ModuleList([Block() for _ in range(BLOCKLAYERS)]),
            ln = LayerNorm(E),
        ))

        self.lm_head = nn.Linear(E, vocabSize, bias=False)
        # Referencia em https://paperswithcode.com/method/weight-tying
        self.transformer.embeddingTable.weight = self.lm_head.weight

        # Inicializar os pesos do modelo
        self.apply(self._init_weights)
        # Por causa das camadas residuais, aplicar um inicializador especial para
        elas
        for pn, p in self.named_parameters():
            if pn.endswith('proj.weight'):
                nn.init.normal_(p, mean=0.0, std=0.02/math.sqrt(2 * BLOCKLAYERS))

        print("Numero de parametros: %.2fM" % (self.get_num_params()/1e6,))

    def forward(self, x, y=None):
        b, c = x.shape

        tok_emb = self.transformer.embeddingTable(x) # (B, C) -> (B, C, E)

```

```

    pos_emb = self.transformer.positionEmbeddingTable(torch.arange(c,
↪dtype=torch.long, device=device)) # (C) -> (C, E)

    x = self.transformer.drop(tok_emb + pos_emb) # (B, C, E) + (C, E) -> (B, C,
↪E)
    for block in self.transformer.blocks:
        x = block(x) # (B, C, E)
    x = self.transformer.ln(x) # (B, C, E)

    logits = self.lm_head(x) # (B, C, V)

    if y == None:
        erro = None
    else:
        # Ajusta a dimensionalidade do logits e do y para o calculo da entropia
        # O calculo de erro espera que o objeto de entrada tenha 2 dimensões:
        # Primeiro o n° de valores previstos
        # Segundo o número de classes/caracteres/tokens
        # Ou seja, batchs*context X vocabSize
        # O Y é um vetor de comprimento igual ao primeiro valor da entrada com
↪cada valor no range do número de classes/caracteres/tokens
        # Ou seja, será o vetor de comprimento batchs*context com valores de 0 a
↪vocabSize-1
        b, c, vocabSize = logits.shape
        logits = logits.view(b*c, vocabSize).to(device)
        y = y.view(b*c)
        erro = nn.functional.cross_entropy(logits, y).to(device)

    return logits, erro

@torch.no_grad()
def get_num_params(self, non_embedding=True):
    """
    Retorna o número de parâmetros do modelo.
    """
    n_params = sum(p.numel() for p in self.parameters())
    if non_embedding: # Remove os positional embeddings da contagem
        n_params -= self.transformer.positionEmbeddingTable.weight.numel()
    return n_params

@torch.no_grad()
def _init_weights(self, module):
    '''Inicializa os pesos das funções Lineares e de Embedding'''
    if isinstance(module, nn.Linear):
        nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if module.bias is not None:
            nn.init.zeros_(module.bias)

```

```

elif isinstance(module, nn.Embedding):
    nn.init.normal_(module.weight, mean=0.0, std=0.02)

@torch.no_grad()
def generate (self, x, newTokens: int):
    # X eh (B, C)
    for _ in range(newTokens):
        # Prediz o proximo caractere do contexto C (baseado no ultimo caractere)
        idx = x[:, -C:] # limita aos ultimos tokens
        logits, _ = self(idx)
        logits = logits[:, -1, :] # (B, C, V) -> (B, V)
        probs = torch.softmax(logits, dim=-1) # (B, V)
        nextChar = torch.multinomial(probs, num_samples=1) # (B, 1)
        x = torch.cat((x, nextChar), dim=1) # (B, C+1)
    return x

#### Funções auxiliares ####
dataDir = 'data'
outDir = 'treino'

## Get data dos arquivos binarios ##
def getDataBatches(tipoDados):
    '''
        Obtêm **B** trecho(s) de tamanho **C** da entrada do **tipoDados** definida
    '''
    if tipoDados == "treino":
        dados = np.memmap(os.path.join(dataDir, 'train.bin'), dtype=np.uint16,
        ↪mode='r')
        #dados = torch.tensor(transformar(textoTreino, charToIndex), dtype=torch.
        ↪long)
    else:
        dados = np.memmap(os.path.join(dataDir, 'test.bin'), dtype=np.uint16,
        ↪mode='r')
        #dados = torch.tensor(transformar(textoTest, charToIndex), dtype=torch.
        ↪long) # textoTest
    n = torch.randint(0, len(dados) - C, (B,)) #Min, Max e quant. de inteiros
    x = torch.stack([torch.from_numpy((dados[i:i+C]).astype(np.int64)) for i in
    ↪n])
    y = torch.stack([torch.from_numpy((dados[i+1:i+1+C]).astype(np.int64)) for i
    ↪in n])

    # Caso não seja CUDA, remove pin_memory
    return x.pin_memory().to(device, non_blocking=True), y.pin_memory().
    ↪to(device, non_blocking=True)

```

```

## Criando o codificador e decodificador ##
def transformar(entrada, padrao):
    '''
    Codifica/Decodifica a entrada baseado no padrão passado.
    '''
    if (type(entrada) == list):
        return ''.join([padrao[c] for c in entrada])
    else:
        return [padrao[c] for c in entrada]

path = os.path.join(dataDir, 'meta.pkl')
vocabSize = None
if os.path.exists(path):
    with open(path, 'rb') as f:
        meta = pickle.load(f)
        vocabSize = meta['vocab_size']
    print(f"Vocabulário encontrado: {vocabSize} caracteres (inside {path})")

### Hiper parametros ###
# N. de Batches (B)
# Tamanho de Contexto (C)
# N. de Embeddings (E)
# HeadSize (H)
hiperParametros = {
    'C': 256, # Números de caracteres utilizados no contexto
    'B': 64, # Quantidade de amostras
    'E': 384, # Dimensão de embedding
    'numberHeads': 6,
    'blockLayers': 6, # Numero de camadas
    'dropout': 0.2,
    'bias': True
}

torch.manual_seed(seed)
torch.cuda.manual_seed(seed)
torch.backends.cuda.matmul.allow_tf32 = True # allow tf32 on matmul
torch.backends.cudnn.allow_tf32 = True # allow tf32 on cudnn

# model
if init_from == 'resume':
    ckpt_path = os.path.join(outDir, 'ckpt.pt')
    checkpoint = torch.load(ckpt_path, map_location=device)
    checkpoint_model_args = checkpoint['hiperParametros']

    for k in ['blockLayers', 'numberHeads', 'B', 'E', 'C', 'bias', 'dropout']:
        hiperParametros[k] = checkpoint_model_args[k]

```

```

B, C, E = hiperParametros['B'], hiperParametros['C'], hiperParametros['E']
numberHeads = hiperParametros['numberHeads']
H = E//numberHeads # head Size

BLOCKLAYERS=hiperParametros['blockLayers']
BIAS=hiperParametros['bias']
DROPOUT=hiperParametros['dropout']

model = LanguageModel(vocabSize)
state_dict = checkpoint['model']

## Erro ja encontrado, mantido por facilidade
# fix the keys of the state dictionary :(
# honestly no idea how checkpoints sometimes get this prefix, have to debug
↪more
unwanted_prefix = '_orig_mod.'
for k,v in list(state_dict.items()):
    if k.startswith(unwanted_prefix):
        state_dict[k[len(unwanted_prefix):]] = state_dict.pop(k)

model.load_state_dict(state_dict)

model.eval()
model.to(device)

# look for the meta pickle in case it is available in the dataset folder
load_meta = False
meta_path = os.path.join('data', 'meta.pkl')
load_meta = os.path.exists(meta_path)

print(f"Loading meta from {meta_path}...")
with open(meta_path, 'rb') as f:
    meta = pickle.load(f)
# TODO want to make this more general to arbitrary encoder/decoder schemes
stoi, itos = meta['stoi'], meta['itos']
encode = lambda s: [stoi[c] for c in s]
decode = lambda l: ''.join([itos[i] for i in l])

# encode the beginning of the prompt
if start.startswith('FILE:'):
    with open(start[5:], 'r', encoding='utf-8') as f:
        start = f.read()
start_ids = encode(start)
x = (torch.tensor(start_ids, dtype=torch.long, device=device)[None, ...])

# run generation

```

```

with torch.no_grad():
    with ctx:
        for k in range(num_samples):
            y = model.generate(x, max_new_tokens)
            print(decode(y[0].tolist()))
            print('-----')

```

Vocabulário encontrado: 137 caracteres (inside data/meta.pkl)

Numero de parametros: 10.70M

Loading meta from data/meta.pkl...

Não era outro descoje; ia eu creio livro perder ciêncio, e deve um mesmo dias
distinto novas, até ao mais de teu correr gesto que suspira desconsigo; nem alei
a feição, tenho menos amorcem de esperanças pontos do tempo e tivera cena e os
coisas; onde sei afendem.

De vi.

Concertasse os voltados.

Brasileiros povos mais o ombjeiço o insinguem.

Entres de passar de meio crianças impressões importar coisas aluciosas do

Frencesso ficaros, mas planos perdentes, e ainda não tinha vinte, e não me se
post

Tínito não se não podem tarde.

Venchia ouses nem primeiros da paixão do gesto, e alcambados que o belo confesso
da minha abastavam até à história complática cão alguma, o aborrido, não
hesitara à impulesidade a deixará daturalmente da mociedade do público
recesseamento; adiga-se que saí lembrar o livro imensado.

- Ah! continuou com - o escrito, direi que não ainda escravo, sem não a mão,
parecia pretendê-lo.

dopenas ós a verdade.

Não há varia no próprio diamante que pergunta o achimador nada, e

Não acontravam-se os olhos repelendos do pretento, e mortava só muito vial e se
casar na pnuca d'imllise; deixou-se que acaba achar que ninguém, a melhor sem
dirig5 de papel excelência, com ele procurasse do pano essador à experdência de
Idéia Da esperança, Goara figura! Não acessiva morte aos dias, algumas e e
encontrava das revoluções erra dias ruas outras anos, - mas o recebite trintaís,
por uma gasta para muita lença perprimeira a oferção da Cilima.

Efim só ainda por fundados.

A confessa da

Não é encante.

- Aqui havia ignior.

- A ernota nada.

Não nem é uma moça proversa: .

os bom; pensa começou crer em fácil, mas os que é um dia e outro ritor não atreve.

Se acudiu.

- Não o céu de um dia para em convenidor.

Quanto o primeiro é ao passador do país de madra, pura que lhe, malhei fazer a diferença, todo novo, levante, e tudo dedes, dez mais recondiva que eu precuso de um dogo, famo guerro fortunho; e saíram ainda sela impossível até caminha alguma vez.

O senhor com que para de um homem
